



Prueba Técnica

Fecha: 29/05/2026

Recomendación: Leer atentamente este documento hasta entender el mecanismo que se desea implementar antes de empezar con su resolución.

La presentación de la resolución de este ejercicio será durante una entrevista técnica, donde se evaluará al postulante mediante preguntas sobre su implementación.

La solución debe poder levantarse de manera local. La resolución de este ejercicio debe estar subido en un repositorio Github público con el nombre: "Prueba Técnica – (Nombre Postulante)"

Objetivo

Desarrollar un asistente automatizado capaz de responder preguntas de soporte utilizando documentación técnica proporcionada.

La solución debe utilizar:

- n8n
 - Python
 - APIs REST
 - Un LLM local o la capa gratuita de la API OpenAI
-

Escenario

La empresa cuenta con documentación interna sobre:

- autenticación
- configuración de servicios
- errores frecuentes
- solución de problemas

Actualmente los agentes responden manualmente preguntas repetitivas. Se requiere construir un asistente que utilice la documentación disponible para responder consultas automáticamente.

Parte 1 — Ingesta de documentación

Como parte de este ejercicio se entregará una carpeta `/docs` con archivos:

- `.txt`
- `.md`
- `.pdf` (opcional)
- `.json`

El postulante debe implementar un proceso que extraiga y normalice contenido:

- leer archivos
 - limpiar texto
 - eliminar ruido
 - dividir contenido en fragmentos utilizables
-

El sistema debe soportar:

- múltiples documentos
- contenido desordenado
- caracteres especiales
- documentos largos

Parte 2 — Workflow en n8n

Implementar un workflow que reciba preguntas de usuarios mediante Webhook HTTP.

Ejemplos de preguntas

`"¿Cómo reinicio el servicio de autenticación?"`

`"El sistema devuelve error 502, ¿qué significa?"`

`"No puedo acceder al dashboard"`

Parte 3 — Recuperación de información

El chatbot debe:

- buscar información relevante en la documentación
 - utilizar únicamente contenido relacionado con la consulta
 - generar una respuesta contextual
-

La respuesta NO debe inventar información que no exista en la documentación.

Si la información no existe:

- debe indicarlo explícitamente
-

Parte 4 — Integración con IA

El sistema debe utilizar OpenAI API

Se evaluará

- calidad del contexto enviado al modelo
- manejo de prompts
- organización del flujo
- eficiencia de recuperación
- claridad de respuestas

Parte 5 — Procesamiento con Python

Python debe encargarse de:

Procesamiento de documentos

Ejemplos:

- chunking
 - limpieza
 - embeddings
 - indexación
 - búsqueda semántica
 - normalización
-

Parte 6 — Manejo de errores

El sistema debe manejar:

- preguntas sin respuesta
- errores de API
- timeouts
- inputs vacíos

Parte 7 — Deployment

La solución debe poder ejecutarse localmente.

Incluir en el repositorio entregado:

- README
 - `.env.example`
 - instrucciones claras
-

Entregables

- workflows n8n exportados
- código fuente(pyhton)
- documentación para el levantamiento y ejecución